

A Retrieval-Augmented Generation System for Knowledge-Grounded Dialogue

Fatma Gizem Yilmaz and Atahan Uz

Bogazici University

Department of Computer Engineering

{gizem.yilmaz1, atahan.uz}@std.bogazici.edu.tr

Abstract

Producing conversational responses that are explicitly grounded in external knowledge requires reliable retrieval and precise passage selection at each stage of the pipeline. We present an end-to-end retrieval-augmented generation system for the Wizard of Wikipedia benchmark (Dinan et al., 2019) with eight pipeline components: a coreference-aware query rewriter, a per-dialogue passage cache, proposition-level dense retrieval, a cross-encoder reranker, generative knowledge selection, query-conditioned passage distillation, active grounding verification, and a dual faithfulness gate. Where the original methods rely on fine-tuned task-specific classifiers, we replace them with zero-shot LLM prompts so the full pipeline runs without local training. An ablation on 21 WoW pairs shows that proposition-level indexing improves Recall@1 by 14.3 points over a paragraph-level baseline. The system runs end-to-end and can be inspected through an interactive web interface that shows every stage’s trace.

1 Introduction

Conversational agents that ground their responses in external knowledge must retrieve the right passage, identify the most informative content within it, generate a response faithful to that content, and verify the result is actually grounded. The Wizard of Wikipedia (WoW) benchmark (Dinan et al., 2019) captures this setting in 22,311 dialogues where every Wizard turn is annotated with the Wikipedia passage it was grounded in.

A naive RAG implementation runs into several failure modes here. Conversational queries carry pronouns and elliptic references that confuse dense retrieval. Embedding a multi-fact paragraph as a single vector dilutes the signal of any one fact. Topically close passages slip through to the generator, and generators then mix retrieved facts with hallucinations. Our system dedicates one component to each of these issues and verifies the output before returning it. The architecture and contributions are described in Section 2; Section 3 then walks through each component in pipeline order.

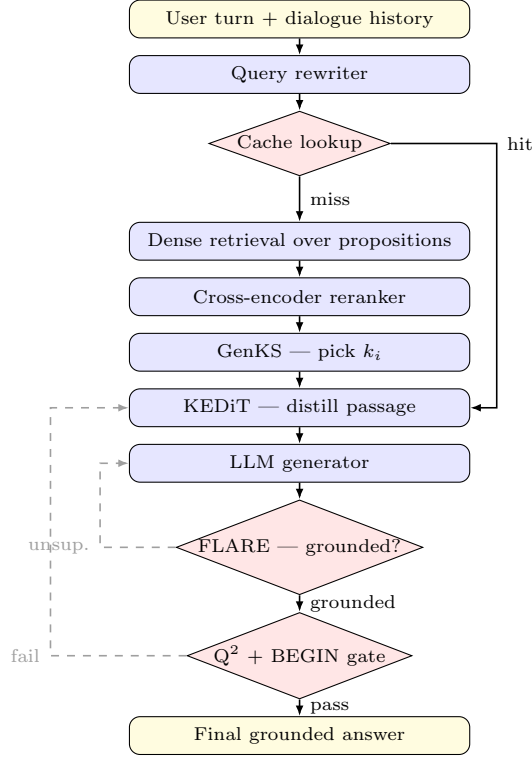


Figure 1: System architecture. Solid arrows: main flow. Dashed arrows: regeneration loops from FLARE (unsupported claims, re-retrieve) and the faithfulness gate (Q^2 or BEGIN failed, regenerate with the GenKS runner-up). The cache lets same-topic follow-up turns bypass dense retrieval, reranking, and selection.

2 System Architecture

The system has the following components, shown in Figure 1:

1. a coreference-aware query rewriter with paraphrases for recall;
2. proposition-level dense indexing following Chen et al. (2023), with a parent-paragraph collapse at query time;
3. a cross-encoder reranker for precision;
4. a per-dialogue passage cache that bypasses retrieval on same-topic follow-up turns;
5. zero-shot LLM reimplementations of GenKS, KEDiT, FLARE, and a Q^2 +BEGIN faithfulness gate, replacing the original fine-tuned classifiers;
6. a Node.js + FastAPI web interface (with a Streamlit fallback) that shows every stage’s trace and per-stage timings.

In pipeline order: the user turn and dialogue history go through the query rewriter, which emits one coreference-resolved rewrite plus two paraphrases. The rewrite is matched against the per-dialogue passage cache; on a hit the cached passage skips directly to KEDiT, otherwise the system runs dense retrieval over a proposition-level index, reranks the top- N candidates with a cross-encoder, and lets GenKS pick exactly one passage. KEDiT distills the chosen passage into a query-conditioned brief, the LLM generator produces a draft answer, and FLARE inspects the draft for unsupported claims, re-retrieving and regenerating once if any are found. A final Q^2 +BEGIN gate scores the answer for faithfulness; if either check fails, the GenKS runner-up is re-distilled and the answer is regenerated once.

3 Pipeline Components

3.1 Conversation-aware Query Rewriting

Dense retrieval is highly literal: “When was it built?” carries no retrievable content until the pronoun is resolved. We use an LLM to rewrite each user turn into a self-contained query conditioned on the dialogue history, then generate $K-1$ paraphrases of the rewrite (default $K=3$). For example, “When was it built?” after a turn about the Eiffel Tower becomes “When was the Eiffel Tower built?”, plus paraphrases like “What year was the Eiffel Tower constructed?”. All K variants are passed to the dense retriever and proposition scores are max-pooled across variants. The cross-encoder reranker only sees the canonical rewrite, since its job is precision rather than recall.

3.2 Per-dialogue Passage Cache

Conversations stay on topic, so the passage chosen for one turn often answers the next one as well. Each dialogue has its own LRU-bounded cache (at most $N=10$ entries) holding the chosen passage, its embedding, the turn it was added on, and its Q^2 score. Before running the main retrieval pipeline we embed the rewritten query and compare it to every cached entry by cosine similarity; if the best match exceeds τ_{cache} (default 0.70), the cached passage goes straight to Stage 3 (KEDiT) and the dense+rerank+GenKS stages are skipped. FLARE’s re-retrieval (Section 7) consults the same cache first.

The cache entry embedding uses the best proposition of the passage rather than the full paragraph: short, focused text matches short queries much better. With this change a follow-up like “What was he famous for?” after a turn about Isaac Asimov reaches similarity 0.754 to the cached Asimov entry, comfortably above τ_{cache} ; the parent-paragraph embedding gave only 0.634, a miss. The cache is an engineering optimization: it does not change the answer the system would produce, only the latency.

3.3 Proposition-level Passage Indexing

A Wikipedia paragraph typically bundles several distinct facts; embedding the whole paragraph as one vector averages those facts and weakens the signal for any specific query. Following Dense X Retrieval (Chen et al., 2023), we decompose each paragraph into atomic propositions before indexing, prompting an LLM to split the paragraph and resolve every pronoun to the article title so each proposition is interpretable in isolation. Chunking is cached by SHA-1 of (title, text) so re-runs cost nothing on the LLM side; malformed output falls back to the full paragraph.

Propositions are embedded by `qwen3-embedding-8b` (4096-dimensional, L2-normalized) and stored in a NumPy matrix. At query time the K query variants are embedded in one batched call; the top- N propositions per variant are pooled by max and collapsed back to top- K *parents* by keeping the highest-scoring proposition per parent. The generator always sees full paragraphs while retrieval benefits from finer granularity. Our experimental index covers 40 representative parent paragraphs (about 457 propositions); the same code path applies unchanged at the full 185,503-paragraph corpus scale.

3.4 Cross-encoder Reranking

A bi-encoder is fast but can promote passages that share vocabulary with the query without being specifically relevant. We pair it with a cross-encoder reranker (Qwen3-Reranker) that scores each candidate by comparing query and passage jointly via cross-attention, producing relevance scores in $[0, 1]$. Both signals are preserved on each result so each stage’s contribution can be analyzed independently. Table 1 shows the effect on two example queries: the cross-encoder confidently keeps the correct passage above 0.99 while collapsing imprecise candidates to near zero, even when their cosine scores were moderately high.

(a) “What is Isaac Asimov famous for?”

Rank	Title	Dense	Rerank
1	Isaac Asimov	0.767	0.9951
2	Science fiction	0.496	0.0001
3	Science fiction film	0.396	0.0000

(b) “Tell me about the Ministry of Magic in Harry Potter”

Rank	Title	Dense	Rerank
1	Ministry of Magic	0.801	0.9986
2	Harry Potter	0.659	0.6297
3	Order of the Phoenix (fictional org.)	0.632	0.0522

Table 1: Dense cosine scores vs. Qwen3-Reranker scores for two example queries. The cross-encoder demotes topically related but imprecise passages to near zero while confirming the target passage above 0.99.

Source	Raw passages	Cumul. unique
train.json	1,188,195	166,262
valid_random_split.json	+63,392	171,027
test_random_split.json	+62,077	175,595
valid_topic_split.json	+62,765	180,291
test_topic_split.json	+62,443	185,503

Table 2: Passage corpus statistics after deduplication across the five WoW split files.

4 Corpus Construction

Our retrieval corpus is derived from the Wizard of Wikipedia dataset (Dinan et al., 2019). For each dialogue turn, we collect the chosen-topic passage and all passages listed in the `retrieved_passages` field, then remove exact (title, paragraph) duplicates across all five split files. The resulting corpus contains 185,503 unique passages. Table 2 shows the raw passage counts and running unique totals as each split is processed. The corpus is built once and treated as a static resource by all pipeline modules.

5 Generative Knowledge Selection (Stage 2)

We follow Sun et al. (2023) and frame knowledge selection as sequence generation: each reranked candidate is given an identifier $k_1, \dots, k_K$ and the model is asked to emit one. The original paper fine-tunes BART-base on WoW; lacking the resources to do that, we use `gpt-oss-120b` zero-shot, passing it the dialogue history, the current turn, and the labelled candidates, and asking for strict JSON with fields `chosen` and `reason`. Outputs outside the allowed identifier set or with malformed JSON fall back to k_1 and are flagged in the interface.

Only the selected passage is passed downstream; the runner-ups stay in the response so that FLARE can re-introduce them during re-retrieval and the faithfulness gate can fall back to the next-best passage when it triggers a regeneration.

6 Parameter-efficient Knowledge-Grounded Generation (Stage 3)

KEDiT (Zhang et al., 2025) compresses the selected passage before passing it to the generator, so the generator does not waste context on irrelevant parts. The original paper uses a BERT + Q-Former bottleneck to compress the passage into $m=16$ learnable vectors and injects them into a frozen Llama-3-8B via a knowledge-aware adapter fine-tuning less than 2% of parameters — both of which need GPU training. We replace this with a zero-shot, query-conditioned extraction: the LLM is given the chosen passage and the user turn and produces a JSON object with a one-sentence `summary` and 2–5 short `facts` that are directly relevant to the question and supported by the passage. The original passage is kept for the faithfulness gate to score against; only the brief is fed to the generator.

For example, for “Who runs the Ministry of Magic?” over the full Ministry of Magic paragraph, distillation extracts just the two leadership facts (“The Minister for Magic is the head of the Ministry”; “Kingsley Shacklebolt takes over after Voldemort’s death”), dropping facts about first appearances, corruption, and so on, and the generator produces a tight focused answer.

7 Active Retrieval during Decoding (Stage 4)

FLARE (Jiang et al., 2023) detects when the generator is producing something it is not sure about and triggers a new retrieval step. The original FLARE-direct variant monitors per-token probabilities below a threshold θ . Our LLM endpoint does not reliably expose per-token logprobs, so we replace that signal with an LLM-based grounding check on the draft: a second call sees the draft and the source passage and lists any specific claims (names, dates, numbers, attributions) not supported by the source, ignoring rephrasings and framing.

If any claims are flagged, those claims plus the user turn become the new retrieval query. We consult the cache (Section 3.2) first; on a miss we run dense retrieval plus reranking on the new query, merge the top result with the originally selected passage (so we keep what GenKS chose), and regenerate once. As a sanity check, on a draft that fabricated “He won the Nobel Prize in Literature in 1972 for his Foundation trilogy” FLARE flagged both fabricated facts and produced a new retrieval query; on a truthful draft over the same passage it returned `grounded=true` and the pipeline passed through unchanged.

8 Faithfulness Verification (Stage 5)

After FLARE we run a final faithfulness gate combining two checks. Q^2 (Honovich et al., 2021) measures factual consistency by generating questions from the response, answering them from the source, and comparing the answers. BEGIN (Dziri et al., 2022) classifies the response as *Fully Attributable*, *Not Attributable*, or *Generic*. The original implementations use four fine-tuned models (T5-base, Albert-XL, two RoBERTas); we replace them with two zero-shot LLM calls running in parallel via `asyncio.gather`. The Q^2 call extracts 2–5 claims, generates a question per claim, answers each from the source, and reports the fraction of matches; the BEGIN call returns a single label plus a one-sentence rationale. Parse failures default to a non-triggering label so a noisy verifier never silently injects retrieval loops.

If $Q^2 < \tau$ or BEGIN says *Not Attributable*, the gate re-runs KEDiT on the GenKS runner-up passage and regenerates the answer once; we do not re-run the gate, to avoid loops. On a faithful Asimov answer the gate reported $Q^2=1.00$ across four QA pairs and BEGIN = *Fully Attributable* and did not trigger. On a hallucinated answer that fabricated a Nobel Prize win and a UN chairmanship, Q^2 collapsed to 0.00 and BEGIN returned *Not Attributable*, correctly firing the regeneration.

9 Results and Ablation

We evaluate retrieval on the 21 (apprentice-query, gold-passage-title) pairs from the WoW *test random* and *valid random* splits whose gold title is one of the 40 parents in our index (9 distinct titles, dominated by *Science fiction* with 9 and *Science fiction film* with 4). For each pair we generate one rewrite plus two paraphrases and retrieve under four conditions. Recall@ K is the fraction of pairs whose gold title appears in the top- K parents.

Effect of proposition indexing. The proposition-level index improves Recall@ K at every K : +14.3 at R@1, +4.8 at R@3, +14.3 at R@5. Encoding a multi-fact paragraph as one vector averages its facts; a focused query matches the centroid less reliably than it matches one of the underlying propositions. Three pairs remain unsolved under any condition (e.g. “What basic topics should I study?” $\rightarrow$ *Oddworld Inhabitants*) because the apprentice text gives almost no retrieval signal, which puts a practical R@5 ceiling near 0.91 on this slice.

Condition	R@1	R@3	R@5
paragraph	0.476	0.762	0.762
paragraph + rerank	0.524	0.762	0.857
proposition (ours)	0.619	0.810	0.905
proposition + rerank	0.524	0.810	0.857

Table 3: Retrieval Recall@ K on 21 WoW pairs whose gold passage is in our index. *paragraph* embeds each parent as a single vector; *proposition* is our proposition-level index with the parent-collapse step. The reranker operates on the dense top-10 pool. Query rewriter output (1 rewrite + 2 paraphrases) is identical across conditions.

Effect of the cross-encoder reranker. The reranker helps the weaker paragraph baseline (+4.8 R@1, +9.5 R@5) where the dense ordering is noisier. On top of the already-strong proposition retriever it is slightly worse at R@1 (−9.5) and unchanged at R@5: on a few queries it promotes a topically related but wrong passage above the correct one. Its precision contribution is intact (Table 1): even when the top- K set does not change, the score gap between the right passage and the distractors widens from cosine 0.5–0.8 to rerank 0.99 vs near zero, which makes the downstream GenKS step easier. The 21-pair slice is small and topically skewed, so the R@1 regression on top of the proposition retriever should be read as suggestive rather than conclusive.

10 System Interfaces

A FastAPI backend exposes `POST /api/run` (full pipeline trace with per-stage timings) and `POST /api/dialogue/reset` (clears the per-dialogue cache). The front-end is a Vite + React + TypeScript single-page app styled with Tailwind that renders each stage as a separate card with inputs, outputs, scores, and timings; clickable example queries and an editable dialogue history make specific behaviors easy to reproduce. A complete pipeline trace as rendered by this interface is shown in Appendix A (Figures A.1–A.5). A simpler Streamlit interface is kept as a fallback; both front-ends call the same backend.

11 Conclusion

We have presented an end-to-end retrieval-augmented generation system for knowledge-grounded dialogue over Wizard of Wikipedia. All eight pipeline components — query rewriting with paraphrases, a per-dialogue passage cache, proposition-level dense retrieval, cross-encoder reranking, GenKS, KEDiT, FLARE, and a Q^2 +BEGIN faithfulness gate — are integrated and exposed through two web interfaces. Where the original methods rely on task-specific fine-tuned classifiers we substitute zero-shot LLM prompts, so the full pipeline runs without GPU training resources. The retrieval ablation in Section 9 gives the first quantitative result: on 21 WoW pairs whose gold passage lies inside our index, proposition-level retrieval reaches 0.619 / 0.810 / 0.905 at Recall@1 / @3 / @5, which is +14.3 / +4.8 / +14.3 points above a paragraph-only baseline. Two follow-up experiments need no code change: a larger quantitative evaluation over the full WoW test splits using standard retrieval and generation metrics, and a Turkish cross-lingual evaluation reusing our multilingual LLM endpoint in place of the original TranslateGemma.

Acknowledgements

We thank the KBB (Kredi Kayıt Bürosu) for providing us access to the endpoints which allows us to use Qwen3-Embedding-8b and GPT-Oss-120b models.

References

Tong Chen, Hongwei Wang, Sihao Chen, Wenhao Yu, Kaixin Ma, Xinran Zhao, Hongming Zhang, and Dong Yu. 2023. Dense X retrieval: What retrieval granularity should we use? In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15159–15177, Singapore. Association for Computational Linguistics.

- Emily Dinan, Stephen Roller, Kurt Shuster, Angela Fan, Michael Auli, and Jason Weston. 2019. Wizard of Wikipedia: Knowledge-powered conversational agents. In *International Conference on Learning Representations*.
- Nouha Dziri, Hannah Rashkin, Tal Linzen, and David Reitter. 2022. Evaluating attribution in dialogue systems: The BEGIN benchmark. *Transactions of the Association for Computational Linguistics*, 10:1066–1083.
- Or Honovich, Leshem Choshen, Roei Aharoni, Ella Neeman, Idan Szpektor, and Omri Abend. 2021. Q2: Evaluating factual consistency in knowledge-grounded dialogues via question generation and question answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3856–3873, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, Singapore. Association for Computational Linguistics.
- Weiwei Sun, Pengjie Ren, and Zhaochun Ren. 2023. Generative knowledge selection for knowledge-grounded dialogues. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 2077–2088, Dubrovnik, Croatia. Association for Computational Linguistics.
- Bo Zhang, Hui Ma, Dailin Li, Jian Ding, Jian Wang, Bo Xu, and HongFei Lin. 2025. Efficient tuning of large language models for knowledge-grounded dialogue generation. *Transactions of the Association for Computational Linguistics*, 13:1007–1031.

A Web Interface: a Full Pipeline Trace

This appendix walks through a complete pipeline run as rendered by the Node.js + FastAPI web interface, for the query “*Tell me about the Ministry of Magic in Harry Potter*” on an empty dialogue history. The five figures below are consecutive screenshots of the same page taken as the user scrolls from the composer at the top down to the final grounded answer at the bottom.

RAG Pipeline Tester dialogue · b279eaa5 New conversation

Type a question or pick an example, then watch each pipeline stage produce its output.

TRY AN EXAMPLE

COREFERENCE →	DIRECT LOOKUP →	TOPIC QUESTION →	THEME QUESTION →
Resolve "he" using prior turns. What is he famous for?	Specific entity, no history. Tell me about the Ministry of Magic in Harry Potter.	Broad genre overview. What is the history of science fiction?	Concept across multiple articles. How is time travel depicted in fiction?

DIALOGUE HISTORY (none) +

› Tell me about the Ministry of Magic in Harry Potter. Run

EXT 3 · CACHE miss sim 0.000 / τ 0.70 1 / 10 cached

STAGE 1 Query rewrite 1015 ms

Rewrite

information about the Ministry of Magic in the Harry Potter series

Paraphrases

details on the Ministry of Magic from Harry Potter

overview of the Ministry of Magic in J.K. Rowling's Harry Potter

Figure A.1: Top of the page: the title bar with the dialogue id chip and the NEW CONVERSATION button; the row of clickable example queries; the (empty) dialogue-history editor; the composer with the user’s turn and the RUN button; the cache banner reporting a miss with similarity 0.000 (the dialogue has no entries yet); and Stage 1 showing the canonical rewrite plus two paraphrases produced by the query rewriter.

STAGE 2 Dense retrieval · pool top-7				138 ms
#	DENSE	TITLE	BEST PROPOSITION	
1	0.801	Ministry of Magic	Ministry of Magic is first mentioned in Harry Potter and the Philosopher's Stone.	
2	0.705	Harry Potter fandom	Harry Potter fandom interacts online.	
3	0.686	Magical objects in Harry Potter	The article Magical objects in Harry Potter contains a list of magical objects used in the Harry Potter series.	
4	0.676	Religious debates over the Harry Potter series	Supporters of the Harry Potter series have said that the magic in the Harry Potter novels is more in the vein of the fairy tale Cinderella.	
5	0.659	Harry Potter	Harry Potter chronicles the life of a young wizard named Harry Potter.	
6	0.632	Order of the Phoenix (fictional organisation)	Order of the Phoenix (fictional organisation) is a secret organisation in the "Harry Potter" series of fiction books written by J. K. Rowling.	
7	0.628	Places in Harry Potter	Places in Harry Potter contains numerous settings for the events in J. K. Rowling's fantasy novels.	

STAGE 3 Cross-encoder reranker · top-3				882 ms
[1]	Ministry of Magic	chosen by GenKS	rerank 0.9986 dense 0.801	
<i>Ministry of Magic is first mentioned in Harry Potter and the Philosopher's Stone.</i> The Ministry of Magic is the government of the Magical community of Britain in "J. K. Rowling's Wizarding World". The magical government in Britain is first mentioned in "Harry Potter and the Philosopher's Stone," the Ministry makes its first proper appearance in "Harry Potter and the Order of the Phoenix". Throughout the books, it is generally depicted as either corrupt, incompetent, or both, with its high officials blind to actual events and dangers. It reaches a zenith of corruption before being effectively taken over by Lord Voldemort. At the end of the final book, following Voldemort's death, Kingsley Shacklebolt takes over the ministry, changing it for the better.				

Figure A.2: Stage 2 (dense retrieval, pool top-7) shows the seven parent paragraphs collapsed from the proposition index together with their cosine scores and the best-matching proposition for each. The first of three Stage 3 cards is shown: the Qwen3 cross-encoder reranker has scored the Ministry of Magic passage at 0.9986 and tagged it as the GenKS choice.

[2]

Harry Potter

rerank 0.6372 dense 0.659

Harry Potter chronicles the life of a young wizard named Harry Potter.

Harry Potter is a series of fantasy novels written by British author J. K. Rowling. The novels chronicle the life of a young wizard, Harry Potter, and his friends Hermione Granger and Ron Weasley, all of whom are students at Hogwarts School of Witchcraft and Wizardry. The main story arc concerns Harry's struggle against Lord Voldemort, a dark wizard who intends to become immortal, overthrow the wizard governing body known as the Ministry of Magic, and subjugate all wizards and muggles, a reference term that means non-magical people. Since the release of the first novel, "Harry Potter and the Philosopher's Stone", on 26 June 1997, the books have found immense popularity, critical acclaim, and commercial success worldwide.

[3]

Order of the Phoenix (fictional organisation)

rerank 0.0540 dense 0.632

Order of the Phoenix (fictional organisation) is a secret organisation in the "Harry Potter" series of fiction books written by J. K. Rowling.

The Order of the Phoenix is a secret organisation in the "Harry Potter" series of fiction books written by J. K. Rowling. Founded by Albus Dumbledore to fight Lord Voldemort and his followers, the Death Eaters, the Order lends its name to the fifth book of the series, "Harry Potter and the Order of the Phoenix". Before the Harry Potter series starts – when the character Lord Voldemort declared war on the wizarding world – Albus Dumbledore, headmaster of Hogwarts School of Witchcraft and Wizardry and an upstanding and powerful citizen of the wizarding world, attempted to take control of the situation by founding the Order of the Phoenix.

STAGE 4 GenKS · knowledge selection

512 ms

k1

Ministry of Magic

reason: k1 directly describes the Ministry of Magic, its role, depiction, and key events, matching the user's request for information about it.

Figure A.3: The remaining two reranked candidates (Harry Potter at 0.6372 and Order of the Phoenix at 0.0540), followed by Stage 4 where GenKS picks identifier k_1 (Ministry of Magic) together with the one-sentence justification produced by the LLM.

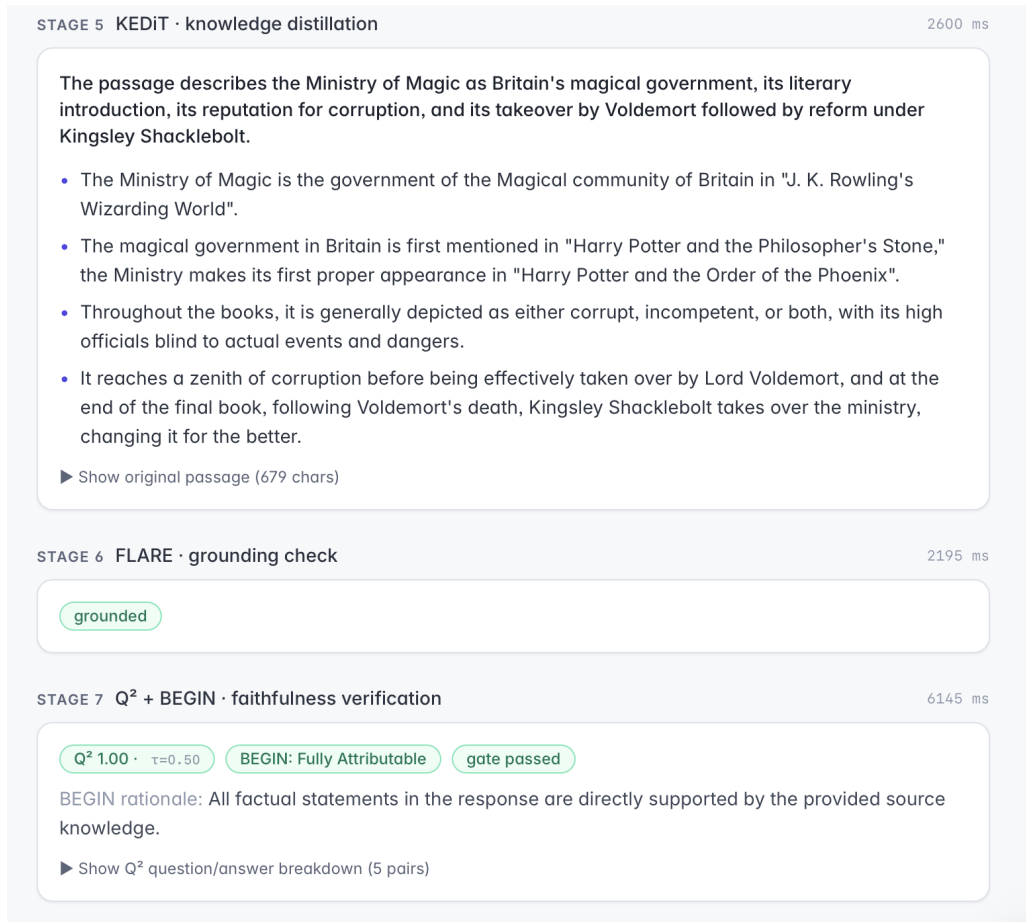


Figure A.4: Stage 5 (KEDiT knowledge distillation) compresses the full Ministry of Magic paragraph into a one-sentence summary plus three query-relevant facts. Stage 6 (FLARE grounding check) returns **grounded** so no re-retrieval is triggered. Stage 7 (Q²+BEGIN faithfulness verification) reports Q²=1.00 across five generated QA pairs and BEGIN=*Fully Attributable*; the gate passes.

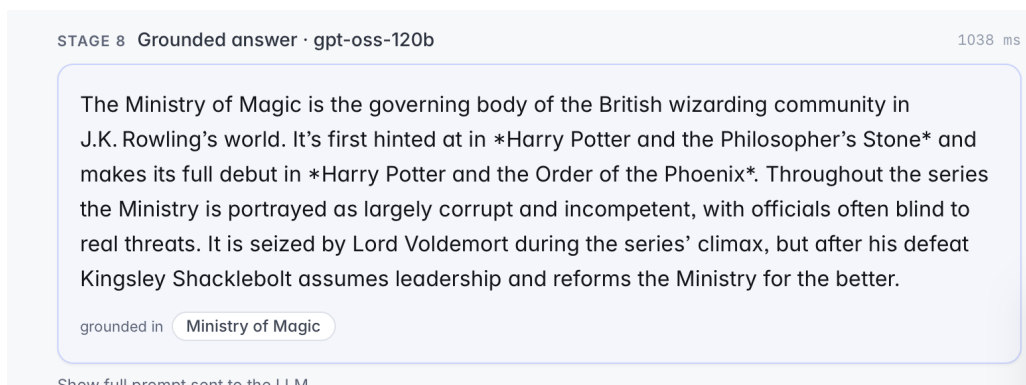


Figure A.5: Stage 8 shows the final grounded answer produced by `gpt-oss-120b` conditioned on the KEDiT brief. The GROUNDED IN chip names the source passage; an expandable control below reveals the full prompt sent to the LLM for inspection.